

1. Guía de comprensión de Flex

Esta guía tiene como objetivo comprobar la comprensión del funcionamiento básico de Flex. Las preguntas deben responderse con palabras propias, utilizando ejemplos cuando sea necesario.

1.1. Función de Flex

1. ¿Qué problema resuelve Flex dentro de un compilador?
2. ¿Qué recibe como entrada Flex?
3. ¿Qué produce como resultado?
4. ¿Cuál es la diferencia entre lo que hace Flex y lo que hará posteriormente Bison?

1.2. Lexemas y tokens

1. ¿Qué es un lexema?
2. ¿Qué es un token?
3. Explicar con un ejemplo la diferencia entre ambos.

Por ejemplo, para:

```
int edad = 123;
```

identificar los lexemas:

```
int
edad
=
123
;
```

y asignarles sus respectivos tokens.

4. ¿Se puede decir que un token es una clasificación del lexema? Explicar por qué.

1.3. Patrones

Considérense las siguientes reglas:

"int"	{ return INT; }
[0-9]+	{ return NUMERO; }
[a-zA-Z_][a-zA-Z0-9_]*	{ return IDENTIFICADOR; }
"+"	{ return SUMA; }

1. ¿Qué es cada parte de una regla como:

```
[0-9]+ { return NUMERO; }
```

Especificar qué representa:

```
[0-9]+
return NUMERO
```

2. ¿Qué relación existe entre un patrón y un lexema?
3. ¿Flex busca palabras concretas o patrones que puedan reconocer partes de la entrada?

1.4. Cómo encuentra un emparejamiento

Considérense las reglas:

```
[0-9]      { return DIGITO; }  
[0-9]+    { return NUMERO; }
```

y la entrada:

1234 + 5

1. Cuando Flex está en el primer carácter, ¿qué intenta hacer?
2. ¿Qué coincidencias puede encontrar?
3. ¿Por qué no devuelve inmediatamente el primer carácter reconocido?
4. ¿Qué significa que Flex busque el emparejamiento más largo?
5. ¿Qué lexema termina reconociendo?
6. ¿Qué ocurre con la posición de análisis después de reconocerlo?

1.5. Maximal munch

Explicar con palabras propias qué significa el principio de *emparejamiento más largo* o *maximal munch*.
Considérense las reglas:

```
"="      { return ASIGNACION; }  
"=="     { return IGUALDAD; }
```

y la entrada:

==

1. ¿Qué patrones coinciden?
2. ¿Cuál gana?
3. ¿Por qué?

1.6. Empates entre patrones

Considérense las reglas:

```
"if"     { return IF; }  
[a-z]+   { return IDENTIFICADOR; }
```

y la entrada:

if

1. ¿Qué reglas coinciden?
2. ¿Qué longitud tiene cada emparejamiento?
3. ¿Cuál gana?
4. ¿Qué ocurre cuando dos reglas producen un emparejamiento de la misma longitud?
5. ¿Qué cambiaría si se invirtiera el orden de las reglas?

1.7. `yytext`

1. ¿Qué es `yytext`?
2. Si Flex reconoce:

`1234`

¿qué contiene `yytext`?

3. Si posteriormente reconoce:

`+`

¿qué contiene `yytext`?

4. ¿`yytext` almacena todos los lexemas encontrados anteriormente?
5. ¿Para qué podría ser útil `yytext` en un compilador? Considerar especialmente números, identificadores y errores léxicos.

1.8. `yyleng`

1. ¿Qué contiene `yyleng`?
2. Si:

`yytext = "1234"`

¿cuánto vale `yyleng`?

3. ¿Cuál es la diferencia entre:

```
yytext
yyleng
yylex()
```

1.9. `yylex()`

1. ¿Qué hace `yylex()`?
2. ¿Qué devuelve?
3. ¿Cuál es la diferencia entre `yytext`, `yyleng` y `yylex()`?
4. Cuando usamos Flex junto con Bison, ¿quién llama normalmente a `yylex()`?

1.10. Entrada no reconocida

Considérense las siguientes reglas:

```
"int"      { return INT; }
[0-9]+     { return NUMERO; }
"+"        { return SUMA; }

. {
    printf("Error léxico: %s\n", yytext);
}
```

y la entrada:

```
int x = 10 @ 20
```

1. ¿Qué ocurre con `int`?
2. ¿Qué ocurre con `10`?
3. ¿Qué ocurre con `@`?
4. ¿Qué contiene `yytext` cuando se ejecuta la regla `.`?
5. ¿Qué ocurre después de procesar `@`?

1.11. Espacios

Considérese la regla:

```
[ \t\n]+ { /* ignorar */ }
```

1. ¿Qué reconoce esta regla?
2. ¿Por qué normalmente no queremos generar tokens para los espacios?
3. ¿Qué ocurre con una entrada como:

```
int      x
```

¿Flex procesa los espacios? ¿Los devuelve como tokens?

1.12. Ejercicio integrador

Considérense las siguientes reglas:

```
"if"           { return IF; }
"=="          { return IGUALDAD; }
"="           { return ASIGNACION; }
[0-9]+         { return NUMERO; }
[a-zA-Z_][a-zA-Z0-9_]* { return IDENTIFICADOR; }
"+"           { return SUMA; }
[ \t\n]+       { /* ignorar */ }
.              { return ERROR; }
```

Analizar la siguiente entrada:

```
if edad == 123 + edad @ 5
```

Completar la siguiente tabla:

Entrada restante	yytext	yytext	Token
if edad == 123 + edad @ 5			
edad == 123 + edad @ 5			
== 123 + edad @ 5			
123 + edad @ 5			
+ edad @ 5			
edad @ 5			
@ 5			
5			

No es necesario incluir las posiciones correspondientes a los espacios que son ignorados.

1.13. Pregunta final

Explicar con palabras propias:

¿Qué hace Flex desde que recibe la entrada `1234 + 56` hasta que termina produciendo los tokens?

La explicación debería incluir el reconocimiento de patrones, el emparejamiento más largo, el uso de `yytext` y `yylen`, la ejecución de las acciones y la continuación del análisis sobre la entrada restante.